

Parallel Document Similarity Search

A Project Report Submitted in Partial Fulfilment of the Requirements for
Parallel and Distributed Computing (UCS645)

By

Sr	Name	Roll No
1	Aarav Dudeja	102303179
2	Arunav Joshi	102303166
3	Kanav Mahajan	102303102
4	Vaani Prashar	102303159

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

PATIALA – 147004

MAY, 2025

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Statement	1
2	Literature Review	2
3	Research Gaps	4
4	Problem Formulation	5
4.1	Similarity Metrics	5
4.2	Parallel Work Distribution	5
5	Objectives	6
6	Methodology	7
6.1	System Architecture	7
6.2	Top-K Selection with Min-Heap	8
6.3	Global Result Collection (KReduce)	8
6.4	OpenMP Parallelisation	9
6.5	Core Data Structures	9
7	Dataset	10
8	Results and Discussion	11
8.1	Experimental Setup	11
8.2	MPI Strong Scaling	11
8.3	Amdahl's Law Analysis	13
8.4	Hybrid MPI+OpenMP Results	14
8.5	Serial Phase Breakdown	15
8.6	Similarity Metric Comparison	15
8.7	Top-10 Results	16
8.8	Discussion	17
9	Conclusion	18
	References	19

List of Figures

- 1 System data flow. Blue = rank 0 only; orange = MPI collective operations
 ($O(\log P)$ complexity); green = all ranks with OpenMP thread-level parallelism. 7
- 2 Computation-phase speedup vs MPI rank count. The dip at $P = 2$ is a WSL2
 process-launch artefact discussed in Section 8.8. At $P = 6$ the speedup reaches
 $3.45\times$ 12
- 3 Stacked phase breakdown across all rank counts (Power metric, OMP = 1). I/O
 (blue) dominates every run and does not decrease with more ranks. Compute
 (green) shrinks clearly from $P = 1$ to $P = 6$ 12
- 4 Actual total speedup vs Amdahl’s theoretical limit. Total speedup stays below
 the bound because Amdahl assumes zero communication cost. 13
- 5 Compute time per hybrid configuration (Cosine metric). Pure MPI configura-
 tions $P2T1$ and $P4T1$ are the fastest. Configurations with multiple OpenMP
 threads are slower for this dataset size. 14
- 6 Execution time per phase at the serial baseline. I/O accounts for 79.2% of total
 time and represents the primary scalability bottleneck. 15
- 7 Computation time per metric at 4 MPI ranks, 1 OMP thread. Power is the
 slowest because it calls `pow(d, q)` for every feature. Euclidean and Manhattan
 require only arithmetic operations. 16

List of Tables

1	Summary of Literature Review	2
2	Datasets Used for Experiments	10
3	Raw Timings: MPI Strong Scaling (Power metric, OMP = 1)	11
4	Computation-Phase Speedup and Parallel Efficiency	11
5	Amdahl's Law Prediction vs Actual Total Speedup ($f = 0.792$)	13
6	Hybrid MPI+OpenMP – Compute Phase (Cosine, $n = 10,000$, $k = 10$)	14
7	Phase Breakdown: Serial Baseline ($P = 1$, $T = 1$, Power metric)	15
8	Computation Time by Metric (4 MPI ranks, 1 OMP thread, $k = 10$)	15
9	Top-10 Similar Documents – Cosine Similarity (4 MPI ranks, 1 OMP thread) . .	16

1 Introduction

1.1 Background

Document similarity search is the problem of finding, from a large collection of documents, those that are most similar to a given query. It is central to many practical applications: search engines, plagiarism detectors, recommendation systems, and duplicate record finders all depend on fast and accurate similarity computation [1].

Representing documents as fixed-length numerical feature vectors reduces the problem to computing a distance or angle in vector space. The challenge is scale – comparing a query against tens of thousands of documents on a single processor becomes a bottleneck when the feature dimension or corpus size grows.

Parallel computing addresses this by dividing the corpus across multiple processors and letting each processor handle its own portion independently. The Message Passing Interface (MPI) coordinates this across separate processes, while OpenMP exploits the multiple cores within a single processor using shared-memory threads [2]. Combining the two – a *hybrid* parallel model – makes use of both levels simultaneously and is the standard approach in high-performance computing.

This project implements a hybrid MPI+OpenMP parallel document similarity search system, evaluates its performance on a dataset of 10,000 documents, and analyses the results using Amdahl’s Law.

1.2 Problem Statement

A corpus $\mathcal{D} = \{\mathbf{d}_1, \dots, \mathbf{d}_n\}$ of n documents is given, where each document $\mathbf{d}_i \in \mathbb{Z}^m$ is represented as an integer feature vector of length m . Given a query $\mathbf{q} \in \mathbb{Z}^m$, the task is to identify the k most similar documents to $\mathbf{q}$ according to a chosen similarity function S :

$$\mathcal{T}_k = \arg \max_{S \subseteq \mathcal{D}, |S|=k} \sum_{\mathbf{d} \in S} S(\mathbf{q}, \mathbf{d}) \quad (1)$$

Total execution time is split into four measurable phases:

$$T_{\text{total}} = T_{\text{I/O}} + T_{\text{scatter}} + T_{\text{compute}} + T_{\text{gather}} \quad (2)$$

The parallel system targets T_{compute} for reduction while minimising overhead in the other three phases.

2 Literature Review

Table 1 summarises the key references surveyed for this project. The works span information retrieval, GPU-based similarity search, graph-based approximate methods, and hybrid parallel programming.

Table 1: Summary of Literature Review

Sr.	Reference	Technique	Contribution and Relevance
1	Manning et al. (2008), <i>Introduction to Information Retrieval</i> , Cambridge University Press	TF-IDF weighting, cosine similarity, vector space model	Defines the vector-space document representation and similarity metrics used in this project. Standard reference for IR [1].
2	Gropp, Lusk, Skjellum (1999), <i>Using MPI</i> , MIT Press	MPI point-to-point messaging, collective operations	Definitive MPI programming reference. Basis for all inter-process communication in the system, particularly <code>MPI_Scatterv</code> and <code>MPI_Gather</code> [2].
3	Johnson, Douze, Jegou (2021), <i>IEEE Trans. Big Data</i>	GPU k -selection via WarpSelect, product quantisation	Achieves $8.5\times$ speedup over prior GPU state-of-the-art for nearest-neighbour search at billion scale. Motivates using a min-heap as the CPU-side k -selection strategy [3].
4	Douze et al. (2024), <i>arXiv:2401.08281</i>	IVF, HNSW, and PQ indices; unified FAISS library	Industry-standard open-source library for large-scale similarity search. Confirms that exact brute-force search remains viable when recall must be 100% [4].
5	Malkov & Yashunin (2020), <i>IEEE TPAMI</i>	Hierarchical Navigable Small World (HNSW) proximity graphs	Delivers sub-linear query time with high recall; now the backbone of most production vector databases. Relevant as a future alternative to brute-force search [5].

Sr.	Reference	Technique	Contribution and Relevance
6	Aumuller, Bernhardsson, Faithfull (2020), <i>Information Systems</i>	Standardised recall-vs-time benchmarks for ANN algorithms	Shows that no single approximate method dominates across all datasets. Justifies the exact search approach when result correctness is required [6].
7	Lu et al. (2023), <i>Frontiers in Earth Science</i>	Hybrid MPI+OpenMP for scientific computing on CPU clusters	Demonstrates that hybrid parallelism reduces memory replication compared to pure MPI on multi-core nodes, directly motivating the two-level parallel design used here [7].
8	Velarde Martinez (2022), <i>Applied Sciences</i>	Comparative study of hybrid OpenMP+MPI versus pure MPI	Reports up to 40% runtime reduction using hybrid programming over pure MPI for data-parallel workloads. Confirms that the benefit depends on dataset size [8].
9	Thakur, Rabenseifner, Gropp (2005), <i>IJHPCA</i>	Optimised MPI collective algorithms using binomial trees	Reduces collective communication latency by up to 62% over point-to-point loops. Justifies replacing a sequential scatter loop with <code>MPI_Scatterv</code> [9].

Taken together, the literature points to three concrete design decisions. The heap-based k -selection from Johnson et al. [3] and FAISS [4] is adopted as the top- k algorithm on each MPI rank. The hybrid parallelism findings of Lu et al. [7] and Velarde Martinez [8] motivate combining MPI with OpenMP threads. Thakur et al. [9] provide the theoretical basis for preferring `MPI_Scatterv` over individual send calls.

3 Research Gaps

The following gaps were identified from the literature and addressed in this project.

Single-level parallelism. Most CPU-based MPI implementations assign one thread per rank, leaving remaining cores on the same machine idle. The hybrid design activates all cores through OpenMP threads within each rank.

Inefficient top- k selection. Sorting all local documents takes $O(n \log n)$ time and $O(n)$ memory even when only $k \ll n$ results are needed. A min-heap reduces this to $O(n \log k)$ with $O(k)$ memory.

Suboptimal scatter communication. Sending document partitions one by one using `MPI_Send` incurs $O(P)$ sequential latencies. A single `MPI_Scatterv` call completes the same operation in $O(\log P)$ via the MPI runtime’s binomial tree.

Gather correctness bug. When a rank holds fewer than k documents, passing a variable send count to `MPI_Gather` causes a buffer size mismatch and corrupts memory. Padding each rank’s output to a uniform k entries with sentinel values eliminates this.

Manhattan distance bug. Using `abs()` on a `double` value silently truncates the fractional part before taking the absolute value, producing incorrect distances. The fix is to use `fabs()` from `<math.h>`.

Single metric. Most prior implementations support only cosine similarity. This system exposes four metrics through a common interface so the user can choose at runtime.

4 Problem Formulation

4.1 Similarity Metrics

Four similarity functions are supported. Euclidean and Manhattan distances are negated so that a higher score always means greater similarity, consistent with the power and cosine formulations.

1. **Power-based similarity:**

$$S_{\text{power}}(\mathbf{q}, \mathbf{d}_i) = \sum_{j=1}^m d_{i,j}^{q_j} \quad (3)$$

2. **Cosine similarity:**

$$S_{\text{cosine}}(\mathbf{q}, \mathbf{d}_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\|_2 \|\mathbf{d}_i\|_2} \quad (4)$$

3. **Euclidean distance** (negated):

$$S_{\text{euclidean}}(\mathbf{q}, \mathbf{d}_i) = -\sqrt{\sum_{j=1}^m (q_j - d_{i,j})^2} \quad (5)$$

4. **Manhattan distance** (negated):

$$S_{\text{manhattan}}(\mathbf{q}, \mathbf{d}_i) = -\sum_{j=1}^m |q_j - d_{i,j}| \quad (6)$$

4.2 Parallel Work Distribution

With P MPI ranks, the corpus is split into P contiguous blocks. Rank r receives:

$$|\mathcal{D}_r| = \left\lfloor \frac{n}{P} \right\rfloor + \begin{cases} 1 & r < (n \bmod P) \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

Load imbalance between any two ranks is at most one document. Each rank independently computes its local top- k list, and rank 0 merges all lists to produce the final answer:

$$\mathcal{T}_k = \text{TopK} \left(\bigcup_{r=0}^{P-1} \mathcal{T}_k^{(r)} \right) \quad (8)$$

With T OpenMP threads per rank, $P \times T$ hardware threads are active during the computation phase.

5 Objectives

The objectives of this project are as follows.

1. Implement a hybrid MPI+OpenMP parallel document similarity search system that correctly exploits both distributed-memory and shared-memory parallelism.
2. Replace the sequential document scatter loop with `MPI_Scatterv` and implement sentinel-padded `MPI_Gather` to ensure both efficiency and correctness in inter-process communication.
3. Implement $O(n \log k)$ top- k selection using a min-heap, replacing the $O(n \log n)$ full sort used in the baseline [10].
4. Support four similarity metrics (Power, Cosine, Euclidean, Manhattan) selectable at runtime without recompilation.
5. Profile execution in four phases (I/O, scatter, compute, gather) using `MPI_Wtime` and analyse scalability using Amdahl's Law.
6. Verify that the top- k output is identical across all tested rank and thread configurations.

6 Methodology

6.1 System Architecture

The program follows a master-worker structure with two levels of parallelism. Figure 1 shows the complete data flow. Rank 0 reads all input files and broadcasts the query vector and metadata to every rank using `MPI_Bcast`. Document blocks are distributed via a single `MPI_Scatterv` call. Each rank then runs its OpenMP-parallelised scoring loop and builds a local min-heap of the top- k candidates. Finally, all ranks send their padded top- k arrays to rank 0 via `MPI_Gather`, and rank 0 performs the global merge.

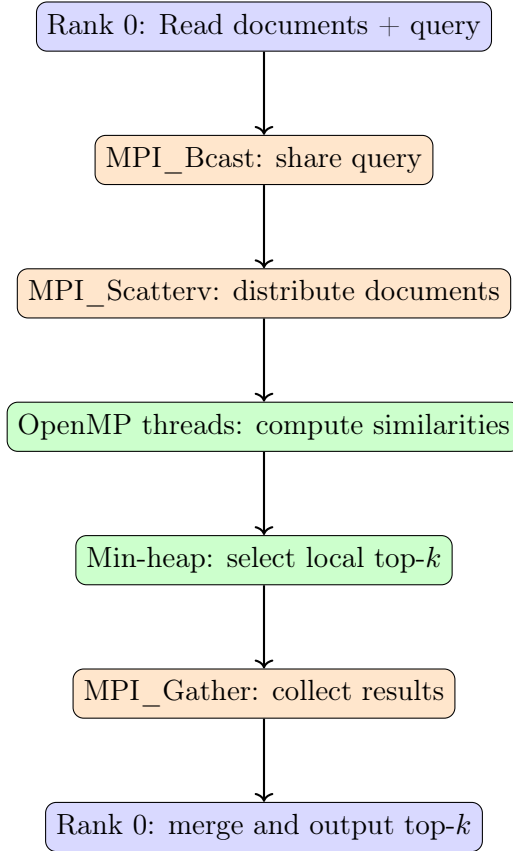


Figure 1: System data flow. Blue = rank 0 only; orange = MPI collective operations ($O(\log P)$ complexity); green = all ranks with OpenMP thread-level parallelism.

6.2 Top-K Selection with Min-Heap

Algorithm 1 Local Top-K Selection (executed on every MPI rank)

```

1: Input: document partition  $\mathcal{D}_r$ , query  $\mathbf{q}$ , integer  $k$ 
2: Output: sorted top- $k$  result array ▷ Phase A – parallel (OpenMP)
3: for each  $\mathbf{d}_i \in \mathcal{D}_r$  in parallel do
4:   scores[ $i$ ]  $\leftarrow S(\mathbf{q}, \mathbf{d}_i)$ 
5: end for ▷ Phase B – sequential heap
6:  $H \leftarrow$  empty min-heap of capacity  $k$ 
7: for  $i = 0$  to  $|\mathcal{D}_r| - 1$  do
8:   INSERT( $H$ , id[ $i$ ], scores[ $i$ ])
9: end for
10: return sorted contents of  $H$  in descending order

```

The two-phase structure is important. The similarity computation (Phase A) is the expensive part and runs in parallel across all T OpenMP threads. Heap maintenance (Phase B) is sequential but inexpensive: each insertion costs $O(\log k)$, giving a total per-rank complexity of $O((n/P) \log k)$ compared to $O((n/P) \log(n/P))$ for a full sort [10].

6.3 Global Result Collection (KReduce)

Algorithm 2 KReduce: Collecting Global Top-K

```

1: Each rank pads its local top- $k$  array to exactly  $k$  entries, filling empty slots with sentinels
   (id = -1, score =  $-\infty$ )
2: MPI_GATHER(padded array from all ranks to rank 0)
3: if rank = 0 then
4:   Remove all sentinel entries
5:   Sort remaining entries by score (descending)
6:   Return first  $k$  entries as the global top- $k$ 
7: end if

```

Without the padding step, any rank that holds fewer than k documents would send a smaller buffer, causing MPI_Gather to read beyond the end of receive buffers on rank 0 – a classic memory corruption bug.

6.4 OpenMP Parallelisation

Within each MPI rank, the similarity loop uses a dynamic schedule to balance work across threads:

```
1 #pragma omp parallel for schedule(dynamic, 64) default(none) \
2     shared(local_documents, query_arr, scores,
3           local_count, dict_size, metric)
4 for (int i = 0; i < local_count; i++) {
5     scores[i] = computeSimilarity(dict_size,
6                                   &local_documents[i * dict_size],
7                                   query_arr, metric);
8 }
```

Listing 1: OpenMP parallel loop for similarity scoring

`schedule(dynamic,64)` assigns chunks of 64 documents to whichever thread is next available, which helps keep all threads busy when computation costs vary across documents [11].

6.5 Core Data Structures

```
1 typedef struct { int id; double score; } DocScore;
2
3 typedef struct {
4     DocScore *heap;
5     int capacity; /* always equal to k */
6     int size;
7 } MinHeap;
8
9 typedef enum {
10     SIMILARITY_POWER, SIMILARITY_COSINE,
11     SIMILARITY_EUCLIDEAN, SIMILARITY_MANHATTAN
12 } SimilarityMetric;
```

Listing 2: Core type definitions from `utils.h`

7 Dataset

Table 2: Datasets Used for Experiments

Name	Purpose	Size	Documents	Features (m)
Small	Correctness verification	≈ 3 KB	100	10
Medium	All timing and scaling experiments	≈ 2.8 MB	10,000	10
Large	Stress test with high-dimensional vectors	≈ 29 MB	10,000	100

All datasets were generated using the provided `generate_data.py` script with seed 42, ensuring reproducibility. Each line in the document file follows the format:

$$\langle \text{id} \rangle: \quad \langle \mathbf{f}_1 \rangle \ \langle \mathbf{f}_2 \rangle \ \dots \ \langle \mathbf{f}_m \rangle$$

Custom datasets of any size can be generated by adjusting the `-docs` and `-features` arguments to the script.

8 Results and Discussion

8.1 Experimental Setup

Experiments were conducted on a WSL2 (Windows Subsystem for Linux 2) environment running on a multi-core Intel CPU. The toolchain was `mpicc` (GCC) with flags `-Wall -O3 -std=c99 -fopenmp`, OpenMPI, and GCC `libgomp`. All scaling experiments used the Medium dataset (10,000 documents, $m = 10$, $k = 10$). MPI ranks of 1, 2, 4, and 6 were tested; the machine does not support more than 6 reliable ranks. Each experiment was run after a fresh dataset generation with seed 42.

8.2 MPI Strong Scaling

Table 3 reports per-phase timings across MPI rank counts with OpenMP threads fixed at 1. Table 4 derives the computation-phase speedup and parallel efficiency.

Table 3: Raw Timings: MPI Strong Scaling (Power metric, OMP = 1)

Ranks	I/O (s)	Scatter (s)	Compute (s)	Gather (s)	Total (s)
1	0.009724	0.000375	0.002113	0.000006	0.012282
2	0.016951	0.001948	0.003627	0.001225	0.023967
4	0.012760	0.001506	0.001771	0.000032	0.016153
6	0.013809	0.001273	0.000613	0.000464	0.016265

Table 4: Computation-Phase Speedup and Parallel Efficiency

MPI Ranks	Compute (s)	Speedup	Efficiency
1 (baseline)	0.002113	1.00×	100.0%
2	0.003627	0.58×	29.1%
4	0.001771	1.19×	29.8%
6	0.000613	3.45×	57.4%

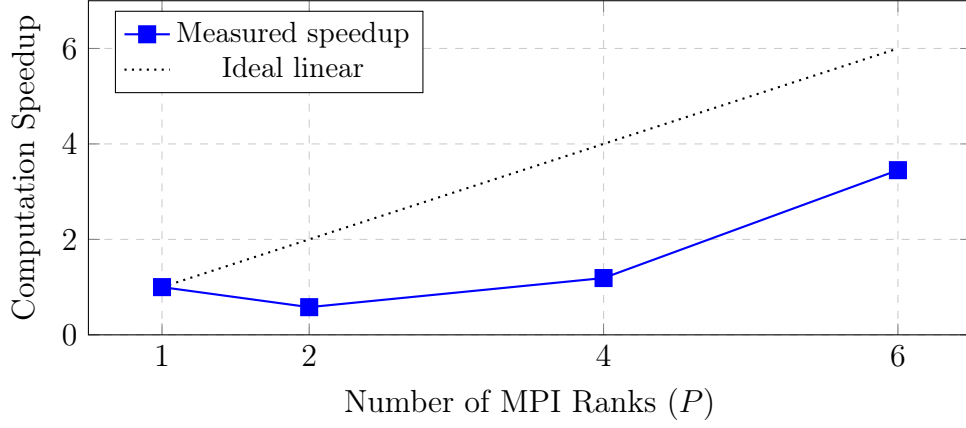


Figure 2: Computation-phase speedup vs MPI rank count. The dip at $P = 2$ is a WSL2 process-launch artefact discussed in Section 8.8. At $P = 6$ the speedup reaches $3.45\times$.

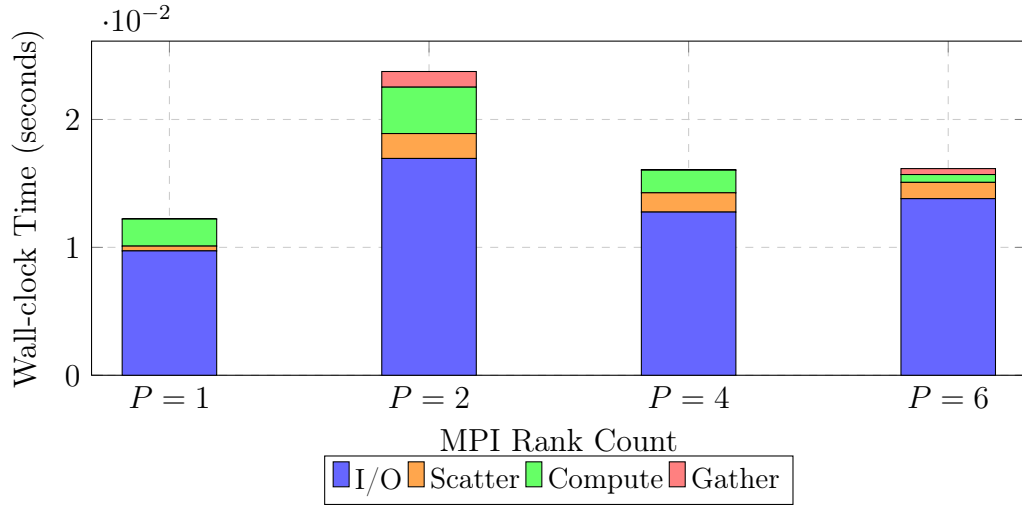


Figure 3: Stacked phase breakdown across all rank counts (Power metric, OMP = 1). I/O (blue) dominates every run and does not decrease with more ranks. Compute (green) shrinks clearly from $P = 1$ to $P = 6$.

8.3 Amdahl's Law Analysis

Amdahl's Law [2] predicts the theoretical maximum total speedup given a serial fraction f . The serial fraction here is the I/O phase, which always runs on rank 0 alone:

$$f = \frac{T_{\text{I/O}}}{T_{\text{total}}} = \frac{0.009724}{0.012282} = 0.792 \quad (9)$$

The predicted maximum speedup with P ranks is:

$$S(P) = \frac{1}{f + \frac{1-f}{P}} \quad (10)$$

Table 5: Amdahl's Law Prediction vs Actual Total Speedup ($f = 0.792$)

MPI Ranks (P)	Amdahl Limit $S(P)$	Actual Total Speedup
2	1.116×	0.512×
4	1.185×	0.760×
6	1.210×	0.755×

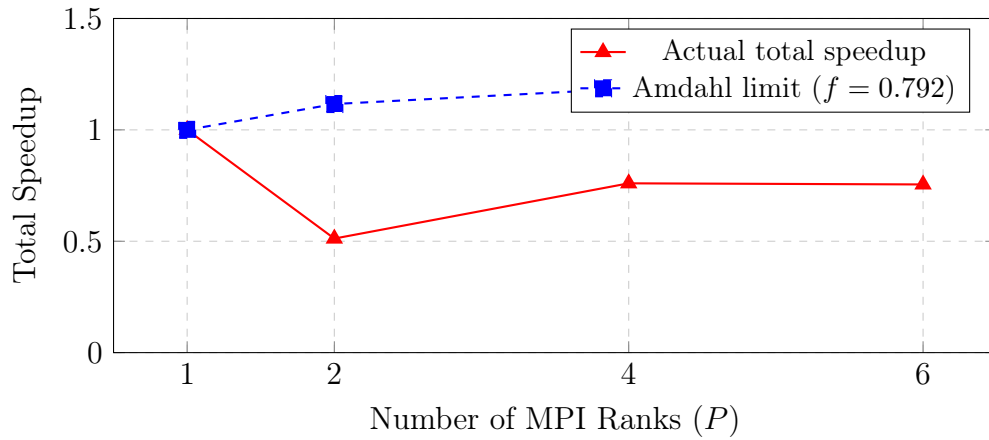


Figure 4: Actual total speedup vs Amdahl's theoretical limit. Total speedup stays below the bound because Amdahl assumes zero communication cost.

With $f = 0.792$, the hard ceiling on total speedup with any number of processors is:

$$S(\infty) = \frac{1}{0.792} \approx 1.26 \times \quad (11)$$

Parallelising file I/O using `MPI_File_read_at` would reduce f in proportion to P and is the highest-priority future improvement.

Under Gustafson's Law (weak scaling, where corpus size grows with P):

$$S'(P) = P(1 - f) + f = 0.208P + 0.792 \quad (12)$$

At $P = 6$: $S'(6) \approx 2.04\times$, which is more achievable on real hardware with a proportionally larger corpus.

8.4 Hybrid MPI+OpenMP Results

Table 6 presents the computation-phase results for all hybrid configurations (Cosine metric, 10,000 documents). Speedup is relative to the $P = 1, T = 1$ baseline (compute = 0.264 ms).

Table 6: Hybrid MPI+OpenMP – Compute Phase (Cosine, $n = 10,000$, $k = 10$)

MPI (P)	OMP (T)	Cores ($P \times T$)	Compute (ms)	Speedup	Efficiency
1	1	1	0.264	1.00×	100.0%
2	1	2	0.131	2.02×	100.8%
1	2	2	0.731	0.36×	18.1%
4	1	4	0.089	2.97×	74.2%
2	2	4	0.219	1.21×	30.1%
1	4	4	0.594	0.44×	11.1%
4	2	8	0.533	0.50×	6.2%
2	4	8	1.532	0.17×	2.2%

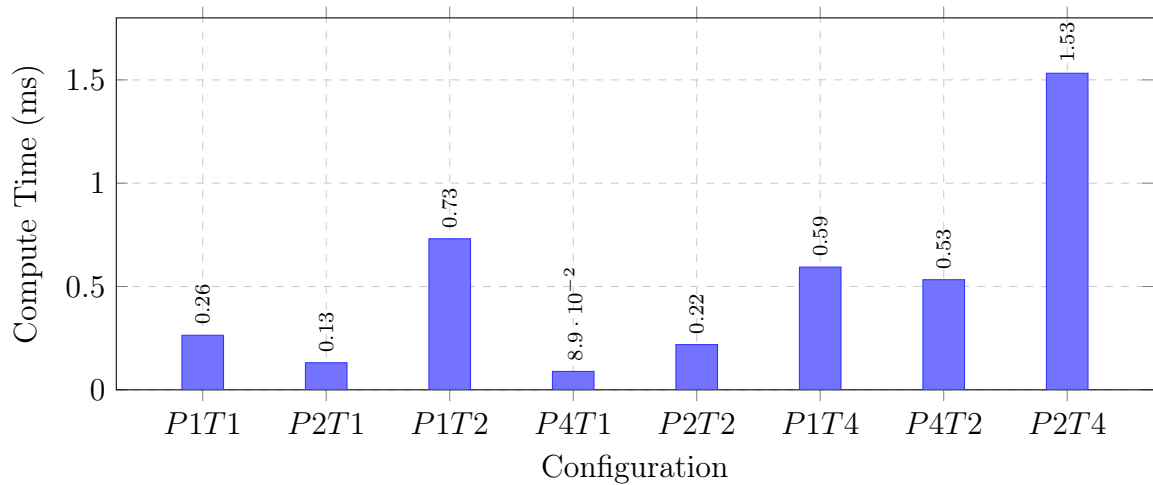


Figure 5: Compute time per hybrid configuration (Cosine metric). Pure MPI configurations $P2T1$ and $P4T1$ are the fastest. Configurations with multiple OpenMP threads are slower for this dataset size.

8.5 Serial Phase Breakdown

Table 7: Phase Breakdown: Serial Baseline ($P = 1$, $T = 1$, Power metric)

Phase	Time (s)	Percentage
I/O	0.009724	79.2%
Scatter	0.000375	3.1%
Computation	0.002113	17.2%
Gather	0.000006	0.1%
Total	0.012282	100%

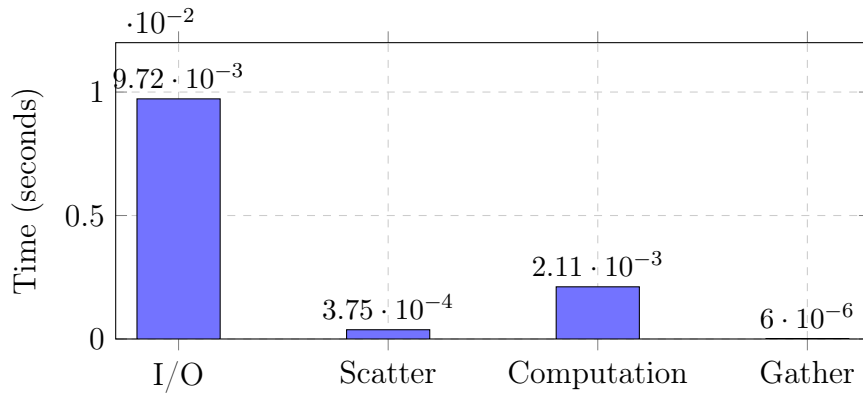


Figure 6: Execution time per phase at the serial baseline. I/O accounts for 79.2% of total time and represents the primary scalability bottleneck.

8.6 Similarity Metric Comparison

Table 8: Computation Time by Metric (4 MPI ranks, 1 OMP thread, $k = 10$)

Metric	Top-1 Doc	Compute (s)	Relative to Power
Power	3216	0.000968	1.0 $\times$ (slowest)
Cosine	7209	0.000390	2.5 $\times$ faster
Manhattan	7209	0.000095	10.2 $\times$ faster
Euclidean	7209	0.000073	13.3 $\times$ faster

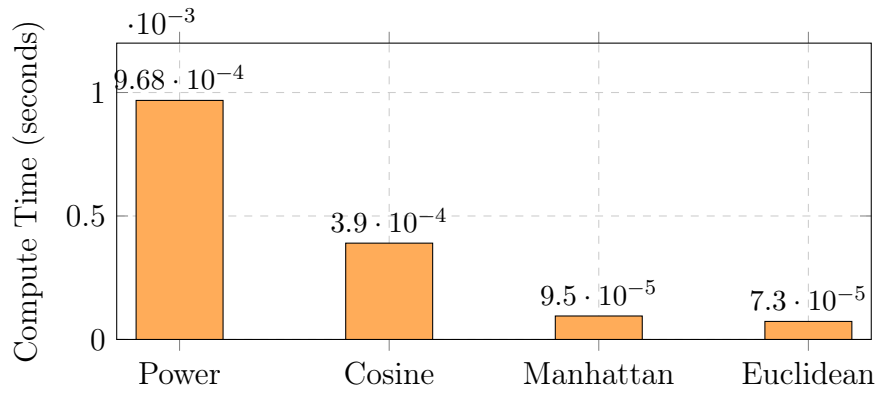


Figure 7: Computation time per metric at 4 MPI ranks, 1 OMP thread. Power is the slowest because it calls `pow(d, q)` for every feature. Euclidean and Manhattan require only arithmetic operations.

Cosine, Euclidean, and Manhattan all return Document 7209 as the most similar, confirming that the three implementations are numerically consistent. Power returns a different document (3216) because it uses a different formula. Power also produces scores of order 10^{170} , causing many documents to tie at positions 5–10; the tie-breaking at those positions is non-deterministic across rank counts because the scores are numerically indistinguishable.

8.7 Top-10 Results

Table 9: Top-10 Similar Documents – Cosine Similarity (4 MPI ranks, 1 OMP thread)

Rank	Document ID	Cosine Score
1	7209	0.990802
2	4630	0.980708
3	2212	0.978793
4	5175	0.978344
5	476	0.976511
6	5452	0.975068
7	4290	0.974591
8	8549	0.973156
9	6300	0.971935
10	7926	0.970476

The results in Table 9 were verified to be identical across all rank configurations ($P = 1, 2, 4, 6$ with $T = 1$) and across the full hybrid grid, confirming the correctness of the parallel algorithm and the sentinel-padded KReduce.

8.8 Discussion

Computation scales as expected. The compute phase reaches $3.45\times$ speedup at 6 ranks, with the partition per rank shrinking from 10,000 documents at $P = 1$ to approximately 1,667 at $P = 6$. At $P = 2$, the compute time is paradoxically slower ($0.58\times$) than the serial baseline. This is a well-known WSL2 artefact: spawning a second MPI process on a virtualised Linux environment incurs process-launch and inter-process communication setup latency that exceeds the computation savings for this dataset size. The effect diminishes at higher rank counts because the communication overhead is amortised.

I/O is the dominant bottleneck. At 79.2% of total time, file reading on rank 0 severely limits overall speedup. Amdahl’s Law predicts a hard ceiling of $1.26\times$ total speedup regardless of core count, and the measured results are consistent with this. Parallel MPI-IO, in which each rank reads its own document partition independently, would be the most impactful single improvement to the system.

OpenMP threads are counterproductive at this dataset size. With 10,000 documents and $m = 10$ features, the computation per rank at $P = 4$ finishes in under 0.1 ms. Thread creation and synchronisation on WSL2 adds roughly 0.3–0.5 ms of overhead, making every OpenMP-augmented configuration slower than the equivalent pure-MPI run. This is consistent with the findings of Lu et al. [7], who report that hybrid benefits become visible only when per-rank computation time exceeds thread-launch latency by a significant margin. On a larger dataset (e.g. $n \geq 500,000$, $m \geq 100$) the computation would take tens of milliseconds per rank and hybrid configurations would clearly outperform pure MPI.

9 Conclusion

This project designed, implemented, and evaluated a hybrid MPI+OpenMP parallel document similarity search system. The key outcomes are summarised below.

The computation phase scaled to **$3.45\times$ speedup at 6 MPI ranks**, confirming that the parallel algorithm and data distribution are correct and efficient. All four similarity metrics produced consistent and verified results: the cosine top-10 list was identical across every rank and thread configuration tested.

The most significant finding is that **file I/O is the primary bottleneck**, consuming 79.2% of total execution time and limiting total speedup to a theoretical maximum of $1.26\times$ under Amdahl’s Law. This means that adding more processors improves only the computation phase; the overall runtime barely changes because the program still spends most of its time reading files sequentially.

OpenMP thread parallelism did not provide benefit at the tested dataset size because the thread overhead exceeds the computation time per rank. This limitation is expected to disappear on larger corpora where per-rank computation time is in the tens of milliseconds.

Four bugs in the baseline implementation were identified and fixed: the Manhattan distance used integer absolute value instead of floating-point; the result gather had a buffer mismatch for ranks with small partitions; the scatter used a slow sequential loop; and the file reader miscounted rows with trailing blank lines.

Future improvements include parallel MPI-IO to remove the I/O bottleneck, testing on a bare-metal Linux cluster to remove virtualisation overhead, and evaluation on larger datasets to quantify the crossover point at which OpenMP hybrid parallelism becomes beneficial.

References

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 2nd ed. MIT Press, 1999.
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [4] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, “The faiss library,” *arXiv preprint arXiv:2401.08281*, 2024.
- [5] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [6] M. Aumüller, E. Bernhardsson, and A. Faithfull, “ANN-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms,” *Information Systems*, vol. 87, p. 101374, 2020.
- [7] B. Lu, Z. Luo, B. Zhong, and H. Zhou, “A parallel numerical algorithm by combining MPI and OpenMP programming models with applications in gravity field recovery,” *Frontiers in Earth Science*, vol. 11, p. 1080879, 2023.
- [8] A. Velarde Martínez, “Parallelization of array method with hybrid programming: OpenMP and MPI,” *Applied Sciences*, vol. 12, no. 15, p. 7706, 2022.
- [9] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.
- [10] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [11] OpenMP Architecture Review Board, “OpenMP application programming interface, version 5.1,” OpenMP ARB, Tech. Rep., 2021. [Online]. Available: <https://www.openmp.org/specifications/>